MKTG 6620 Case 3

AI Session Log and Disclosure

Prepared for the AI-Disclosures repository at the user's request.

Conversation coverage (UTC): 2026-10-20T01:41:39.380Z to 2026-10-24T01:40:25.824Z

Transcript: 27 user prompts and 73 visible assistant messages. Timestamps are recorded UTC times; local dates may differ.

Contents: factual session summary; principal command record; chronological transcript. The complete embedding script as originally displayed is preserved in the transcript.

Scope and source

This disclosure was prepared from the saved conversation for MKTG 6620 Case 3. It includes all actual user prompts and recorded visible assistant replies from the initial assignment-reading request through the request to export this PDF. Original spelling, code, and reported results are retained; only line wrapping and page layout are changed. The summary below is an editorial account of the session, not the student's personal analysis. No missing dialogue has been invented. System/developer instructions, environment-context updates, internal reasoning, and raw tool calls/results are excluded. Commands and results repeated in visible assistant replies remain included. Export-operation messages after the final request are outside the transcript cutoff.

Environment and data inspection

The agent read the assignment recipe and Q1 requirements, checked supplied filenames, Python, and Windows, and created a local project outside OneDrive at C:\Users\bedna\mktg6620\case3. The environment used Python 3.13.15, CPU torch 2.11.0+cpu, sentence-transformers 5.4.1, transformers 4.57.6, and NumPy 2.4.4. Hardware was a Microsoft Surface Laptop 4 with AMD Ryzen 5 Microsoft Surface Edition, 6 physical cores, 12 logical processors, about 8 GB RAM, and Windows 11 Home. The agent counted 42,994 products, 2,852 blank/missing product classes, and 79 development / 200 validation / 200 test queries. Development labels: 4,623 Exact, 23,222 Partial, 8,736 Irrelevant. Validation labels: 11,134 Exact, 63,712 Partial, 29,615 Irrelevant.

Q1: embeddings and rankings

The agent created embed_and_rank.py and displayed its code before execution. The user explicitly reviewed it and authorized running it. The recipe used sentence-transformers/all-MiniLM-L6-v2 revision 1110a243fdf4706b3f48f1d95db1a4f5529b4d41 and BAAI/bge-small-en-v1.5 revision 5c38ec7c405ec4b44b94cc5a9bb96e735b38267a. Products used unchanged search_text. BGE queries received the prescribed prefix, including its final space. Both models ran on CPU with four threads and normalized float32 vectors. Each produced 42,994 x 384 product vectors, requiring 66,038,784 raw bytes. Ranking used dot products against every product and numeric product-ID tie breaking. The final CSV had 9,580 ranking rows. The agent retrieved top-five results for development query 1 without a personal relevance assessment.

Timing measurements and important corrections

The first completed run measured MiniLM product/query encoding at 198.42 / 1.39 seconds and BGE-small at 539.63 / 5.81 seconds. A later rerun was explicitly requested and replaced the diagnostics and rankings. It measured MiniLM product/query encoding at 337.27 / 1.81 seconds and BGE-small at 155,986.92 / 4.96 seconds. The approximately 43-hour BGE elapsed time was reported as anomalous; computer sleep or suspension was suggested but not

established. The production estimate later used the user-specified earlier 539.63-second measurement. Other issues included sandbox command-startup failures, a corrected PowerShell quoting error, an interrupted create-and-run attempt, nonfatal model-cache/download warnings, and a nonfatal invalid-escape docstring warning. The first embedding wrapper returned 1 despite completed outputs and no traceback; the later run explicitly captured Python exit code 0.

Test-label safeguards and the fingerprint workaround

Initially the agent omitted labels_test.csv from the local copy to honor the user's prohibition on accessing it. The development scorer failed because its startup fingerprint loop reads every manifest file, including the missing test-label file. The agent disclosed and used an in-memory change that skipped only that file's fingerprint check; score.py and the manifest stayed unchanged. This workaround was used for development scoring and selected show/format-check commands where test-label access was prohibited. The development keyword baseline was 0.743. Later the user expressly authorized restoring the original test-label file from VD3_student_files.zip without displaying or parsing its contents and allowed normal automatic fingerprinting. The agent then ran the original scorer normally for validation. Test judgments were not used to build either embeddings or the hybrid.

Q2: validation, user selection, and one-time test

Validation nDCG@10 was keyword 0.6750, MiniLM 0.7357, and BGE-small 0.7390. Both embedding methods matched the recipe tolerance. BGE-small validation Exact@1 was 0.5200; its paired nDCG difference versus keyword was +0.0641 with 95% interval +0.0278 to +0.1002. The agent retrieved query 0 rankings, labels, and existing scores for the student's own arithmetic. The user chose BGE-small and supplied the exact reason for selection.json. The agent saved that reason without choosing on the user's behalf. At the user's explicit instruction, it checked that selection was valid, score.py matched the ZIP, and no prior test output or logged test run was found, then scored the test exactly once. Test nDCG@10 was keyword 0.6846 and BGE-small 0.7100; the paired difference was +0.0254 with 95% interval -0.0022 to +0.0527. No subsequent test scoring was requested or performed in the recorded session.

Q3: factual validation-only collection

The agent sorted stored validation per-query scores and retrieved the six largest differences. BGE-small wins: query 168 printers (+1.000000), 345 oriental vanity (+0.926636), and 68 nautical platters (+0.842560). Keyword wins: query 219 trinaic towel rod (-0.930569), 360 huntsville sectional sofa (-0.694765), and 416 wayfair coffee table (-0.583433). It presented keyword/BGE top-ten products and supplied labels, plus factual label-count differences, without assigning failure types. For Q3b it reported judged@10 of 0.8905 for keyword and 0.8805 for BGE-small and retrieved ten unjudged BGE results from ten different validation queries, leaving relevance judgments to the user. Earlier in Q1 the agent did supply brief explanations for one pair of labelled examples; those original replies are preserved in the transcript.

Q4: hybrid experiment and production arithmetic

At the user's instruction, hybrid_rank.py fused existing keyword and BGE top-ten lists with reciprocal rank fusion: sum 1/(60 + rank), descending total score, lower numeric product ID for ties. It used no labels or query text to build the method and generated 4,790 hybrid_rrf rows for all 479 query IDs. Validation-only scoring versus BGE-small gave hybrid nDCG@10 0.7182, judged@10 0.8935, paired difference -0.0209, interval -0.0392 to -0.0034, 61 better and 83 worse queries. Fusion took 0.226 seconds, required no additional encoding or raw vectors, and wrote a 107,976-byte ranking CSV. The agent did not alter the selected model. For 2,000,000 BGE products, it calculated 3,072,000,000 raw bytes (3,072 MB; 2.861 GiB), and linear encoding time of 25,102.57 seconds (about 6 hours 58 minutes 23 seconds), using 539.63 seconds for 42,994 products. It provided factual re-embedding, index-check, and monitoring considerations without a personal business recommendation.

Files and authorship boundaries

The agent created code, rankings, diagnostics, scoring outputs, and the user-authorized selection record. The user repeatedly asked that answers.md remain unchanged and retained responsibility for personal judgments, hand calculations, interpretation, and recommendations. This PDF is a disclosure record, not a submission memo. Transcript passages are the original replies, including any caveats or mistakes; summary statements above identify the relevant corrections.

Principal commands recorded during the session

These are commands actually used, with repetitive full interpreter paths shortened where noted. This list is a summary, not a dump of internal tool traffic. The transcript retains the commands as shown to the user.

```
py --version; python --version; py -0p

py -3.13 -m venv "C:\Users\bedna\mktg6620\case3\.venv"

& '.\.venv\Scripts\python.exe' -m pip install --upgrade pip

& '.\.venv\Scripts\python.exe' -m pip install -r requirements.txt

& '.\.venv\Scripts\python.exe' -m pip install 'torch==2.11.0' --index-url
https://download.pytorch.org/whl/cpu

& '.\.venv\Scripts\python.exe' -m pip install 'torch==2.11.0' 'sentence-transformers==5.4.1'
'transformers==4.57.6'

& '.\.venv\Scripts\python.exe' -m pip check

python score.py score --split dev  [initial failure; completed through disclosed in-memory fingerprint
skip]

python embed_and_rank.py  [executed after review, then rerun at explicit user request]

python score.py check --rankings my_rankings.csv  [first through in-memory skip; later original scorer
after authorized restore]

python score.py score --split validation --rankings my_rankings.csv

python score.py show --query-id 0 --rankings my_rankings.csv  [in-memory test-label fingerprint skip]

python score.py select --method bge_small --reason "Although it has a longer encoding time, BGE-small
achieved the highest validation nDCG@10 (0.7390) and Exact@1 (0.5200) of the three methods."

python score.py score --split test --selection selection.json --rankings my_rankings.csv  [once]

python score.py show --query-id QUERY_ID --rankings my_rankings.csv  [IDs 168, 345, 68, 219, 360, 416;
in-memory fingerprint skip]

python hybrid_rank.py

python score.py score --split validation --rankings my_rankings.csv my_change.csv --baseline bge_small
--out outputs_q4
```

Chronological conversation transcript

User prompts and visible assistant replies, in recorded order. Message text is unedited apart from typographic line wrapping. Markdown is retained as text. Internal system instructions, reasoning, raw tools, and environment updates are not included.

001 | Turn 01 | USER PROMPT

2026-10-20T01:41:39.380Z

```
Read VD3_QUESTIONS.md, especially the recipe and Q1 requirements. Do not open, read, preview, or analyze
data/labels_test.csv, and do not run test scoring.
```

002 | Turn 01 | ASSISTANT UPDATE

2026-10-20T01:41:42.601Z

```
I'll locate and read `VD3_QUESTIONS.md`, focusing on the recipe and Q1 requirements. I'll leave
`data/labels_test.csv` untouched and won't run test scoring.
```

003 | Turn 01 | ASSISTANT REPLY

2026-10-20T01:41:54.960Z

```
I couldn't read `VD3_QUESTIONS.md` because the local command tool failed to start. Please provide the
file's contents so I can review the recipe and Q1 requirements.

I have not accessed `data/labels_test.csv` or run test scoring.
```

004 | Turn 02 | USER PROMPT

2026-10-20T01:42:18.059Z

```
"C:\Users\bedna\OneDrive\Documents\AI-Class\VD3_student_files"
```

005 | Turn 02 | ASSISTANT UPDATE

2026-10-20T01:42:21.974Z

```
I'll look for `VD3_QUESTIONS.md` in that folder and read the instructions, leaving `data/labels_test.csv`
 untouched.
```

006 | Turn 02 | ASSISTANT REPLY

2026-10-20T01:42:53.425Z

```
Read `VD3_QUESTIONS.md`, including the recipe and Q1.

The recipe requires both pinned model revisions, unchanged `search_text`, the exact BGE query prefix, CPU
 encoding, normalized float32 vectors, and dot-product rankings with lower product IDs breaking ties.
Output must contain ten results per query for all 479 queries and both methods.

Q1 requires:

- **1a:** Product counts, blank classes, queries by split, dev/validation label counts, and one
development query's Exact/Irrelevant examples.
- **1b:** Model details, vector shapes and lengths, raw bytes, encoding times, and computer details.
- **1c:** Each model's top five for one development query, package versions, final script, and AI record.

The assignment reserves judgments and explanations for you to write. It also instructs you to work from a
 local folder outside OneDrive.

I did not access `data/labels_test.csv` or run test scoring.
```

007 | Turn 03 | USER PROMPT

2026-10-20T01:56:07.909Z
First, inspect the folder structure and tell me whether all required assignment files are present. Check my Python version and operating system. I need Python 3.11–3.14.

008 | Turn 03 | ASSISTANT UPDATE

2026-10-20T01:56:15.663Z
I'll check the required filenames, Python version, and operating system without opening any test labels or running scoring.

009 | Turn 03 | ASSISTANT REPLY

2026-10-20T01:57:48.827Z
All required supplied assignment files are present:

```text
VD3_student_files/
├── VD3_QUESTIONS.md
├── VD3_QUESTIONS.html
├── requirements.txt
├── score.py
├── SOURCE.md
└── data/
    ├── products.csv
    ├── queries.csv
    ├── labels_dev.csv
    ├── labels_validation.csv
    ├── labels_test.csv
    ├── keyword_rankings.csv
    ├── data_manifest.json
    └── WANDS_LICENSE.txt
```

This confirms file presence only, not contents or integrity.

- **Python:** 3.13.15 for both `python` and `py`—meets the required **3.11–3.14** range.
- **Operating system:** Windows 11 Home, 64-bit, version 10.0.26200.
- **Location:** The folder is currently in OneDrive. The assignment requires working from a local folder outside OneDrive.

I only listed the test-label filename; I did not open or analyze it, and no scoring was run.

010 | Turn 04 | USER PROMPT

2026-10-20T01:58:01.463Z
Then help set up a local .venv virtual environment and install the dependencies exactly as specified in the assignment: requirements.txt, CPU PyTorch 2.11.0, sentence-transformers 5.4.1, and transformers 4.57.6. Use the appropriate commands for my operating system.

011 | Turn 04 | ASSISTANT UPDATE

2026-10-20T01:58:08.630Z
I'll copy the assignment files to `C:\Users\bedna\mktg6620\case3`, create a `.venv`, and run the assignment's Windows installation commands with the exact package versions. I'll leave `labels_test.csv` untouched and won't run scoring.

012 | Turn 04 | ASSISTANT UPDATE

2026-10-20T01:59:56.929Z
The local `.venv` is created and NumPy 2.4.4 is installed. Pip has downloaded the CPU build of PyTorch 2.11.0 and is installing it; the pinned sentence-transformers and transformers packages are next. The local copy excludes `labels_test.csv`.

013 | Turn 04 | ASSISTANT UPDATE

2026-10-20T02:00:51.551Z
PyTorch is still installing and hasn’t reported an error. Once installation finishes, I’ll verify the package versions, CPU build, and dependency compatibility.

014 | Turn 04 | ASSISTANT UPDATE

2026-10-20T02:02:45.559Z
PyTorch 2.11.0+cpu installed successfully. Pip is now installing sentence-transformers 5.4.1, transformers 4.57.6, and their dependencies.

015 | Turn 04 | ASSISTANT UPDATE

2026-10-20T02:04:57.699Z
Installation is complete, and `pip check` found no dependency conflicts. I corrected a quoting error in my verification command; the final import and version check is now running.

016 | Turn 05 | USER PROMPT

2026-10-20T02:05:20.796Z
report the actual installed versions of Python, torch, sentence-transformers, transformers, and NumPy.

017 | Turn 05 | ASSISTANT REPLY

2026-10-20T02:05:32.895Z
Verified in `C:\Users\bedna\mktg6620\case3\.venv`:

Package	Installed version
Python	3.13.15
torch	2.11.0+cpu
sentence-transformers	5.4.1
transformers	4.57.6
NumPy	2.4.4

All match the assignment requirements. PyTorch is CPU-only, imports succeeded, and `pip check` found no dependency conflicts.

018 | Turn 06 | USER PROMPT

2026-10-20T02:14:56.695Z
run the development keyword baseline: python score.py score and confirm the keyword nDCG@10 is approximately 0.743

019 | Turn 06 | ASSISTANT UPDATE

2026-10-20T02:15:40.106Z
I’ll run the keyword baseline with `--split dev` and check whether nDCG@10 is approximately 0.743.

020 | Turn 06 | ASSISTANT UPDATE

2026-10-20T02:16:15.040Z
The scorer stopped because its startup integrity check tries to read every data file, including the excluded test labels. No test labels were read. I’ll run it with a temporary in-memory adjustment that skips only the test-label fingerprint check, leaving the scoring calculations and supplied files unchanged.

021 | Turn 06 | ASSISTANT REPLY

2026-10-20T02:16:28.723Z

Confirmed: the development keyword baseline has **nDCG@10 = 0.743** across **79 queries**, matching the assignment.

Results saved to `outputs/dev_metrics.csv` and `outputs/dev_per_query.csv` in the local `case3` folder.

The run skipped only the test-label fingerprint check through an in-memory adjustment. The supplied scorer stayed unchanged; no test labels were read and no test scoring ran.

022 | Turn 07 | USER PROMPT

2026-10-20T02:17:57.035Z
Read the CSV files and report: total number of products, number of queries in each split, number of extract, partial and irrelevant labels in development, number of extract, partial and irrelevant lavels in validation.

023 | Turn 07 | ASSISTANT UPDATE

2026-10-20T02:18:01.131Z
I’ll count the products, queries by split, and Exact, Partial, and Irrelevant labels in development and validation. I’ll leave the test-label file untouched.

024 | Turn 07 | ASSISTANT REPLY

2026-10-20T02:18:15.519Z
Total products: 42,994

Query split	Number of queries
Development	79
Validation	200
Test	200
Total	**479**

Label	Development	Validation
Exact	4,623	11,134
Partial	23,222	63,712
Irrelevant	8,736	29,615
Total	**36,581**	**104,461**

These counts came directly from the CSVs. Test labels were not accessed.

025 | Turn 08 | USER PROMPT

2026-10-20T02:20:55.528Z
pick one development query, quote one product labelled extract and one labelled irrevant and say in a sentence why each label fits

026 | Turn 08 | ASSISTANT UPDATE

2026-10-20T02:21:01.435Z
I’ll find one development query and quote an Exact and an Irrelevant product. The assignment asks you to write the sentences explaining why the labels fit, so I’ll provide the examples for you to assess.

027 | Turn 08 | ASSISTANT REPLY

2026-10-20T02:21:58.494Z
Development query **421: “dining room tables”**

Label	Product ID	Product name	Class
Exact	214	“guillot dining table”	Dining Tables

| Irrelevant | 2726 | “patty seat cushion” | Furniture Cushions |

The Exact label fits because the product is a dining table, matching what the shopper requested. The Irrelevant label fits because a seat cushion is a seating accessory rather than the requested table.

The assignment asks you to write these explanations yourself, so use the verified examples to express your own reasoning.

028 | Turn 09 | USER PROMPT

2026-10-20T02:25:40.703Z
Count how many rows in data/products.csv have a blank or missing product_class. Report the exact count.

029 | Turn 09 | ASSISTANT UPDATE

2026-10-20T02:25:57.711Z
I’ll count rows where `product_class` is missing, empty, or contains only whitespace.

030 | Turn 09 | ASSISTANT REPLY

2026-10-20T02:26:15.620Z
2,852 rows in `data/products.csv` have a blank or missing `product_class`, counting whitespace-only values as blank.

031 | Turn 10 | USER PROMPT

2026-10-20T02:27:27.199Z
xplain the test-label fingerprint issue encountered when running the development baseline:

What exact error occurred?

Why was an in-memory adjustment needed?

What part of the scorer's behavior was bypassed?

Did the adjustment affect any scoring calculations or development labels?

Can the development baseline be run normally without this adjustment?

032 | Turn 10 | ASSISTANT REPLY

2026-10-20T02:27:37.206Z
1. **Exact error:** The command failed with:
   ```text
   FileNotFoundError: [Errno 2] No such file or directory:
   'C:\\Users\\bedna\\mktg6620\\case3\\data\\labels_test.csv'
   ```
   This occurred in `load_data()`, where `sha()` calls `Path(p).read_bytes()` to calculate file fingerprints.

2. **Why an in-memory adjustment was needed:** I had excluded `labels_test.csv` from the local copy to honor your instruction. The scorer checks fingerprints for **every file listed in the manifest**, even when scoring development only. It therefore stopped before scoring.

3. **What was bypassed:** Only the fingerprint check for `labels_test.csv`. The temporary adjustment added:
   ```python
   if name == 'labels_test.csv': continue
   ```
   inside the manifest-check loop. Other file checks still ran. The supplied `score.py` and manifest remained unchanged on disk.

4. **Effect on scoring:** None. Development labels, rankings, metric formulas, and scoring calculations

were unchanged. The resulting development keyword **nDCG@10 was 0.743 across 79 queries**. No test-label contents were read.

5. **Can it run normally?** **Not in the current local folder**, because the test-label file is absent. Restoring that file would allow the ordinary command to proceed, but the scorer would then read its bytes for fingerprinting—even with `--split dev`. Under your instruction prohibiting test-label access, the unmodified scorer therefore requires this adjustment.

033 | Turn 11 | USER PROMPT

2026-10-20T02:34:13.556Z
provide me another development query, quote one product labelled exact and one irrelevant.

034 | Turn 11 | ASSISTANT UPDATE

2026-10-20T02:34:21.009Z
I'll retrieve another development query with one Exact and one Irrelevant product from the supplied labels.

035 | Turn 11 | ASSISTANT REPLY

2026-10-20T02:34:33.211Z
Development query **434: "bathroom lighting"**

Label	Product ID	Product name	Product class
Exact	210	"3-light dimmable brushed nickel vanity lighting vanity light"	Vanity Lighting
Irrelevant	5	"vogan 33 '' single bathroom vanity"	Vanities

These are the supplied development labels.

036 | Turn 12 | USER PROMPT

2026-10-20T02:43:38.262Z
please run the python embed_and_rank.py and record the following for each model: method, model id, revision (first 7 characters), query prefix, product vectors (rows x dimensions), number format, smallest and largest vector length, raw bytes, and encoding time and computer

037 | Turn 12 | ASSISTANT UPDATE

2026-10-20T02:43:43.330Z
I'll check whether `embed_and_rank.py` exists and matches the assignment recipe, then run it and record the measured vector details and encoding times. I won't access test labels or run scoring.

038 | Turn 12 | ASSISTANT UPDATE

2026-10-20T02:43:57.007Z
`embed_and_rank.py` is not present, so I'll create it using the exact recipe. Your computer is a Surface Laptop 4 with an AMD Ryzen 5 processor, 6 cores, 12 logical processors, and about 8 GB RAM. I'll use four CPU threads, matching the assignment's timing example, and save the measurements to a JSON file.

039 | Turn 13 | USER PROMPT

2026-10-20T02:46:48.679Z
Since embed_and_rank.py does not exist yet, please create it now using the exact recipe in VD3_QUESTIONS.md.

You may use four CPU threads and save embedding diagnostics to a JSON file.

Before creating the script, verify the exact model IDs, revisions, BGE query prefix, normalization, float32 format, and ranking requirements against the assignment.

Write a readable, commented embed_and_rank.py that generates my_rankings.csv for both models and all 479 queries.

IMPORTANT:

Create the script but DO NOT execute it.

Do not download or load the models yet.

Do not run any scoring commands.

Do not open or inspect test labels.

Do not modify answers.md or score.py.

Once the script is written, show me the code and summarize how it satisfies the assignment's requirements. Stop there so I can review it before execution.

040 | Turn 13 | ASSISTANT UPDATE

2026-10-20T02:46:54.143Z
I'll verify the recipe in `VD3_QUESTIONS.md`, then create the script for review without executing it or loading models. I'll also check whether the interrupted command left a script or process behind.

041 | Turn 13 | ASSISTANT UPDATE

2026-10-20T02:47:07.327Z
The recipe confirms the two pinned revisions, the exact BGE prefix including its trailing space, normalized float32 vectors on CPU, and dot-product ranking with lower product IDs breaking ties. No script or Python process remained from the interrupted attempt. I'll now write the script without running it.

042 | Turn 13 | ASSISTANT REPLY

2026-10-20T02:47:58.340Z
Created [embed_and_rank.py](C:/Users/bedna/mktg6620/case3/embed_and_rank.py). **It has not been executed.** No models were downloaded or loaded, no labels were accessed, and `answers.md` and `score.py` were not modified.

Verified against the assignment:

Requirement	Implementation
MiniLM	Exact model ID and revision `1110a2…`
BGE-small	Exact model ID and revision `5c38ec7…`
Product text	Unchanged `search_text`
Query text	Original query; exact prefix added only for BGE
Embeddings	CPU, four threads, normalized float32
Rankings	Dot product against every product; descending score, lower numeric product ID breaks ties
Output	Both models, all 479 queries, ten results each: **9,580 rows**
Diagnostics	Shapes, lengths, bytes, encoding times, versions, and computer information in JSON

The script separates product and query encoding times and excludes model downloading/loading from those timings.

```python
"""Create the assignment's rankings and embedding diagnostics.

Run manually after review: .\.venv\Scripts\python.exe embed_and_rank.py
Reads only products.csv and queries.csv. No labels or scoring are used.
"""
import csv
import json
import os
```

```
import platform
import time
from importlib.metadata import version
from pathlib import Path

# Set CPU thread limits before importing the numerical libraries.
os.environ['OMP_NUM_THREADS'] = '4'
os.environ['MKL_NUM_THREADS'] = '4'
os.environ['OPENBLAS_NUM_THREADS'] = '4'

import numpy as np
import torch
from sentence_transformers import SentenceTransformer
from threadpoolctl import threadpool_limits

HERE = Path(__file__).resolve().parent
MODELS = (
    {
        'method': 'minilm',
        'model_id': 'sentence-transformers/all-MiniLM-L6-v2',
        'revision': '1110a243fdf4706b3f48f1d95db1a4f5529b4d41',
        'query_prefix': '',
    },
    {
        'method': 'bge_small',
        'model_id': 'BAAI/bge-small-en-v1.5',
        'revision': '5c38ec7c405ec4b44b94cc5a9bb96e735b38267a',
        # The final space is part of the assignment's query prefix.
        'query_prefix': 'Represent this sentence for searching relevant passages: ',
    },
)


def read_csv(filename):
    """Read a named input as UTF-8; preserve text values exactly."""
    with (HERE / 'data' / filename).open(newline='', encoding='utf-8-sig') as file:
        return list(csv.DictReader(file))


def encode(model, texts):
    """Time encoding only, excluding model download/loading and ranking."""
    start = time.perf_counter()
    vectors = model.encode(
        texts,
        batch_size=32,
        device='cpu',
        normalize_embeddings=True,
        convert_to_numpy=True,
        show_progress_bar=True,
        precision='float32',
    )
    seconds = time.perf_counter() - start
    vectors = np.asarray(vectors, dtype=np.float32)
    if not np.isfinite(vectors).all():
        raise ValueError('Embeddings contain nonfinite values.')
    return vectors, seconds


def vector_details(vectors):
    lengths = np.linalg.norm(vectors, axis=1)
    if not np.allclose(lengths, 1.0, atol=1e-5, rtol=0):
        raise ValueError('Embedding lengths are not approximately one.')
    return {
        'shape': list(vectors.shape),
        'number_format': str(vectors.dtype),
        'smallest_vector_length': float(lengths.min()),
```

```
        'largest_vector_length': float(lengths.max()),
        'raw_bytes': int(vectors.nbytes),
    }


def main():
    torch.set_num_threads(4)
    # Sorting numeric IDs also makes file output deterministic.
    products = sorted(read_csv('products.csv'), key=lambda row: int(row['product_id']))
    queries = sorted(read_csv('queries.csv'), key=lambda row: int(row['query_id']))
    if len(products) != 42994 or len(queries) != 479:
        raise ValueError('Expected 42,994 products and 479 queries.')
    product_ids = np.array([int(row['product_id']) for row in products], dtype=np.int64)
    if len(set(product_ids)) != len(products):
        raise ValueError('Duplicate product IDs.')
    if len({row['query_id'] for row in queries}) != len(queries):
        raise ValueError('Duplicate query IDs.')
    product_texts = [row['search_text'] for row in products]

    diagnostics = {
        'computer': {
            'operating_system': platform.platform(),
            'processor': platform.processor(),
            'logical_processors': os.cpu_count(),
            'cpu_threads_used': 4,
        },
        'versions': {'Python': platform.python_version(), **{
            name: version(name) for name in
            ('torch', 'sentence-transformers', 'transformers', 'numpy')
        }},
        'models': [],
    }
    output = HERE / 'my_rankings.csv'
    temporary = HERE / 'my_rankings.csv.tmp'
    # Publish the final CSV only after both models finish successfully.
    with threadpool_limits(limits=4), temporary.open('w', newline='', encoding='utf-8') as file:
        writer = csv.writer(file)
        writer.writerow(['query_id', 'method', 'rank', 'product_id'])
        for config in MODELS:
            print(f"Loading {config['method']}: {config['model_id']}", flush=True)
            model = SentenceTransformer(
                config['model_id'], revision=config['revision'], device='cpu',
            )
            # Both products and queries use this same model instance.
            product_vectors, product_seconds = encode(model, product_texts)
            query_texts = [config['query_prefix'] + row['query'] for row in queries]
            query_vectors, query_seconds = encode(model, query_texts)
            if product_vectors.shape != (42994, 384) or query_vectors.shape != (479, 384):
                raise ValueError('Unexpected embedding shapes.')
            record = {
                **config,
                'revision_first_7': config['revision'][:7],
                'device': 'cpu',
                'products': vector_details(product_vectors),
                'queries': vector_details(query_vectors),
                'product_encoding_seconds': product_seconds,
                'query_encoding_seconds': query_seconds,
                'total_encoding_seconds': product_seconds + query_seconds,
            }
            print(json.dumps(record, indent=2), flush=True)
            diagnostics['models'].append(record)
            (HERE / 'embedding_diagnostics.json').write_text(
                json.dumps(diagnostics, indent=2) + '\n', encoding='utf-8',
            )
            # Compare each query with EVERY product. Full sorting avoids
            # dropping a lower-ID product when scores tie at rank ten.
```

```
            for query, vector in zip(queries, query_vectors):
                scores = product_vectors @ vector
                top_ten = np.lexsort((product_ids, -scores))[:10]
                for rank, index in enumerate(top_ten, start=1):
                    writer.writerow([
                        query['query_id'], config['method'], rank, int(product_ids[index]),
                    ])
            file.flush()
            del model, product_vectors, query_vectors
    temporary.replace(output)
    print(f'Wrote {output}: {479 * 2 * 10:,} ranking rows.', flush=True)


if __name__ == '__main__':
    main()
```

043 | Turn 14 | USER PROMPT

2026-10-20T02:49:49.311Z
I have reviewed embed_and_rank.py, and the implementation appears to match the assignment's recipe.

Please execute the existing script using the project's virtual environment:

..venv\Scripts\python.exe embed_and_rank.py

Do not rewrite or replace the script unless an execution error requires a correction. If an error occurs, explain it before making significant changes.

044 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:49:56.061Z
I'll run the existing script with `.\.venv\Scripts\python.exe`, monitor its progress, and report the measured diagnostics. I won't run scoring or access test labels.

045 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:50:44.728Z
MiniLM has loaded and is encoding the catalog on CPU. Hugging Face reported cache and download warnings, but encoding is proceeding normally. No script changes were needed.

046 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:51:39.844Z
MiniLM has encoded about 27% of the products. The run is progressing without errors; the script will encode the queries and record the diagnostics after the catalog finishes.

047 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:53:15.887Z
MiniLM has encoded about 76% of the catalog. The script remains unchanged and is using the prescribed CPU settings.

048 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:53:56.098Z
MiniLM finished: its product vectors are **42,994 × 384 float32**, with lengths from **0.999999881 to 1.000000119**. Encoding took **198.42 seconds for products** and **1.39 seconds for queries**. BGE-small is now loading.

049 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:56:41.719Z
BGE-small is encoding the catalog and has reached about 25%. It is running more slowly than MiniLM on this computer, with no errors reported.

050 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:57:37.861Z
BGE-small has reached about 37% of the catalog. Encoding speed is varying, so I'll wait for the measured completion time rather than rely on the progress bar's estimate.

051 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:58:41.737Z
BGE-small has passed 44% of the catalog and is still progressing without errors. MiniLM's diagnostics are saved; the final rankings CSV will be published after BGE-small finishes.

052 | Turn 14 | ASSISTANT UPDATE

2026-10-20T02:59:31.937Z
BGE-small has passed halfway through the catalog. When it finishes, I'll verify the rankings file's row counts and rank structure without scoring it.

053 | Turn 14 | ASSISTANT UPDATE

2026-10-20T03:01:08.543Z
BGE-small is about 75% through the catalog. The script has remained unchanged, and no scoring commands have been run.

054 | Turn 14 | ASSISTANT UPDATE

2026-10-20T03:02:01.761Z
BGE-small has reached about 82%. The remaining steps are query encoding, ranking, and saving the final CSV.

055 | Turn 14 | ASSISTANT UPDATE

2026-10-20T03:02:55.260Z
BGE-small has reached about 93% of the catalog. I'll check the saved diagnostics and rankings structure once the process exits.

056 | Turn 14 | ASSISTANT UPDATE

2026-10-20T03:03:26.863Z
Both models finished, and the log reports that all 9,580 ranking rows were written. The command wrapper returned exit code 1 despite that completion message, so I'm checking the output structure and log for errors before calling the run complete. I haven't changed or rerun the script.

057 | Turn 14 | ASSISTANT REPLY

2026-10-20T03:03:49.409Z
The existing script completed both models without changes.
[my_rankings.csv](C:/Users/bedna/mktg6620/case3/my_rankings.csv) contains **9,580 rows**, verified as 479 queries per model, ranks 1–10, with no duplicate products within a query.

Method	Model ID	Revision	Query prefix
minilm	`sentence-transformers/all-MiniLM-L6-v2`	`1110a24`	None
bge_small	`BAAI/bge-small-en-v1.5`	`5c38ec7`	`Represent this sentence for searching relevant passages: `

The BGE prefix includes its trailing space.

Method	Product vectors	Format	Minimum length	Maximum length	Raw product bytes	Product encoding	Query encoding
minilm	42,994 × 384	float32	0.999999881	1.000000119	66,038,784	198.42 s	1.39 s
bge_small	42,994 × 384	float32	0.999999881	1.000000119	66,038,784	539.63 s	5.81 s

Computer: Microsoft Surface Laptop 4, AMD Ryzen 5 Microsoft Surface Edition, approximately 8 GB RAM, Windows 11 Home; four CPU threads used. Timings exclude model downloading/loading and ranking.

Measurements are saved in [embedding_diagnostics.json](C:/Users/bedna/mktg6620/case3/embedding_diagnostics.json).

The command wrapper returned exit code 1, but the log contains the completion message and no traceback; the output structure passed verification. No scoring ran, and no test labels were accessed.

058 | Turn 15 | USER PROMPT

2026-10-20T03:10:28.420Z
The embedding script has finished and generated diagnostics.

Please complete the remaining Q1c data collection.

1. Confirm that my_rankings.csv contains exactly 9,580 ranking rows.
2. Confirm whether python score.py check --rankings my_rankings.csv passed.
3. For development query 1, "smart coffee table", extract the five highest-ranked products from MiniLM and BGE-small using my_rankings.csv and data/products.csv.
4. Report the rank, product ID, product name, and product class for each result.
5. Include the actual software versions and computer information recorded during execution.

Do not interpret whether the products make sense. I will do that myself.

Do not open test labels, run test scoring, or edit answers.md.

059 | Turn 15 | ASSISTANT UPDATE

2026-10-20T03:10:34.114Z
I'll verify the row count, collect each model's top five for query 1, and read the recorded environment details. I'll also run the format checker with the disclosed test-label fingerprint skip; the unmodified command cannot complete while that file is excluded.

060 | Turn 15 | ASSISTANT REPLY

2026-10-20T03:15:48.396Z
Confirmed: `my_rankings.csv` contains exactly **9,580 ranking rows**, excluding the header.

The scorer's format check **passed**, reporting `Format OK.` Both models cover all 79 development, 200 validation, and 200 test queries. As previously disclosed, only the test-label fingerprint check was skipped in memory; `score.py` remained unchanged. The unmodified command cannot complete while `labels_test.csv` is excluded.

For development query **1: "smart coffee table"**, the extracted results are:

MiniLM (`minilm`)

Rank	Product ID	Product name	Product class
1	33698	smart coffee table with storage	Coffee & Cocktail Tables
2	295	buchheit coffee table	Blank
3	294	aalst coffee table	Blank
4	39692	calmar coffee table	Blank
5	40418	dev coffee table	Coffee & Cocktail Tables

BGE-small (`bge_small`)

Rank	Product ID	Product name	Product class
1	33698	smart coffee table with storage	Coffee & Cocktail Tables
2	34828	metre solid wood coffee table	Blank
3	34711	coffee table with storage	Blank
4	5132	solid coffee table	Coffee & Cocktail Tables
5	26951	solid coffee table	Coffee & Cocktail Tables

“Blank” denotes an empty `product_class` value.

Actual versions recorded during execution:

Software	Version
Python	3.13.15
torch	2.11.0+cpu
sentence-transformers	5.4.1
transformers	4.57.6
NumPy	2.4.4

Execution diagnostics recorded Windows `Windows-11-10.0.26200-SP0`, processor `AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD`, 12 logical processors, and **four CPU threads used**. The earlier hardware inspection identified a **Microsoft Surface Laptop 4**, AMD Ryzen 5 Microsoft Surface Edition, with approximately **8 GB RAM**.

No test labels were accessed, no scoring ran, and `answers.md` was not edited.

061 | Turn 16 | USER PROMPT

2026-10-20T03:16:22.649Z
please run the python embed_and_rank.py and record the following for each model: method, model id, revision (first 7 characters), query prefix, product vectors (rows x dimensions), number format, smallest and largest vector length, raw bytes, and encoding time and computer

062 | Turn 16 | ASSISTANT UPDATE

2026-10-20T03:16:27.019Z
I’ll rerun the existing script in the project’s virtual environment and report the measurements from this run. It will replace the rankings and diagnostics files; I won’t change the script, access test labels, or run scoring.

063 | Turn 16 | ASSISTANT UPDATE

2026-10-20T03:17:39.275Z
The rerun has started. I’m logging standard output and warnings separately and capturing Python’s exit code explicitly, so the completion status will be clearer than in the previous run.

064 | Turn 16 | ASSISTANT UPDATE

2026-10-20T03:18:31.814Z
MiniLM is now encoding the catalog. Startup and the first batches are slower than in the previous run; I’ll report the new measured times once this run finishes.

065 | Turn 16 | ASSISTANT UPDATE

2026-10-20T03:19:17.982Z
MiniLM has reached about 7% of the catalog. It is progressing without errors, and the script and thread settings are unchanged.

066 | Turn 16 | ASSISTANT UPDATE

2026-10-20T03:20:12.385Z

MiniLM has passed 13% of the catalog. This run is slower than the first, so the new timing measurements will be useful for showing how runtime can vary on the same computer.

067 | Turn 16 | ASSISTANT UPDATE

2026-10-21T22:54:02.488Z
The latest log shows query encoding has finished. I'll check the process exit status and read the saved diagnostics to confirm whether both models completed.

068 | Turn 16 | ASSISTANT REPLY

2026-10-21T22:54:17.402Z
The rerun completed with **Python exit code 0** and generated **9,580 ranking rows**.

Method	Model ID	Revision (first 7)	Query prefix
minilm	`sentence-transformers/all-MiniLM-L6-v2`	`1110a24`	None
bge_small	`BAAI/bge-small-en-v1.5`	`5c38ec7`	`Represent this sentence for searching relevant passages: `

The BGE prefix includes a trailing space.

Method	Product vectors	Format	Smallest length	Largest length	Raw bytes
minilm	42,994 × 384	float32	0.999999881	1.000000119	66,038,784
bge_small	42,994 × 384	float32	0.999999881	1.000000119	66,038,784

Method	Product encoding	Query encoding	Total encoding
minilm	337.27 s	1.81 s	339.08 s
bge_small	155,986.92 s	4.96 s	155,991.88 s

Timing caveat: BGE's recorded elapsed time is approximately **43.33 hours**. This is an anomalous measurement that may include computer sleep or suspension; the cause is unverified. The previous completed run measured **539.63 seconds** for BGE product encoding.

Computer: Microsoft Surface Laptop 4, AMD Ryzen 5 Microsoft Surface Edition, approximately 8 GB RAM, Windows 11; **four CPU threads used**.

Results are saved in [embedding_diagnostics.json](C:/Users/bedna/mktg6620/case3/embedding_diagnostics.json) and [my_rankings.csv](C:/Users/bedna/mktg6620/case3/my_rankings.csv). No test labels were accessed and no scoring ran.

069 | Turn 17 | USER PROMPT

2026-10-22T01:14:57.127Z
The embeddings have already been generated, and my_rankings.csv has passed the format check. Do not rerun the embedding models.

Please complete the following:

For development query 1, "smart coffee table", retrieve the top five ranked products for both minilm and bge_small from my_rankings.csv.

Create a Markdown table for each method containing:

Rank

Product ID

Product name

Product class

Verify and report the actual installed versions of:

Python

torch

sentence-transformers

transformers

NumPy

Summarize factually what the AI agent did during Q1, including:

Environment setup

Dataset inspection and counting

Creation of embed_and_rank.py

Embedding generation

Ranking generation

Format validation

List the commands executed, important errors encountered, and corrections made. Include the earlier missing labels_test.csv fingerprint issue and its in-memory workaround.

Confirm the complete final embed_and_rank.py is available for inclusion under the Code section of answers.md.

Prepare a factual AI record containing the prompts and key agent responses from the actual session. Do not invent missing prompts or results.

IMPORTANT:

Do not write my personal assessment of whether the search results are sensible.

Do not write my explanation of what I personally checked.

Do not modify answers.md.

Do not inspect test labels or run validation or test scoring.

Do not move on to Q2.

Present everything clearly so I can use it to complete Q1c myself.

070 | Turn 17 | ASSISTANT UPDATE

2026-10-22T01:15:02.165Z
I’ll retrieve the top-five results from the existing rankings, verify the installed versions, and confirm the final script is available. Then I’ll assemble a factual Q1 record from this session, including the commands, errors, and workarounds. I won’t rerun embeddings, score results, or edit `answers.md`.

071 | Turn 17 | ASSISTANT REPLY

2026-10-22T01:16:27.153Z
The existing rankings contain **9,580 rows**, excluding the header. No embedding models were rerun for this request.

Development query 1: “smart coffee table”

MiniLM (`minilm`):

Rank	Product ID	Product name	Product class
1	33698	smart coffee table with storage	Coffee & Cocktail Tables
2	295	buchheit coffee table	Blank
3	294	aalst coffee table	Blank
4	39692	calmar coffee table	Blank
5	40418	dev coffee table	Coffee & Cocktail Tables

BGE-small (`bge_small`):

Rank	Product ID	Product name	Product class
1	33698	smart coffee table with storage	Coffee & Cocktail Tables
2	34828	metre solid wood coffee table	Blank
3	34711	coffee table with storage	Blank
4	5132	solid coffee table	Coffee & Cocktail Tables
5	26951	solid coffee table	Coffee & Cocktail Tables

“Blank” represents an empty `product_class` field. These tables were retrieved from the current `my_rankings.csv` and joined to `data/products.csv`.

Verified installed software and computer

Versions were checked again in the project’s virtual environment using Python’s version information and installed package metadata, without loading either model.

Software	Actual installed version
Python	3.13.15
torch	2.11.0+cpu
sentence-transformers	5.4.1
transformers	4.57.6
NumPy	2.4.4

Computer detail	Recorded information
Computer	Microsoft Surface Laptop 4
Processor	AMD Ryzen 5 Microsoft Surface Edition
Physical cores / logical processors	6 / 12
RAM	8,002,056,192 bytes, approximately 8 GB
Operating system	Windows 11 Home, 64-bit, version 10.0.26200
Embedding execution	CPU, four threads

The runtime diagnostics identify the processor as `AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD`. The computer model, processor marketing name, RAM, and Windows edition came from the earlier hardware inspection.

Factual summary of the agent’s Q1 work

Activity	What the agent did
Environment setup	Inspected the supplied folder and Python installation. Created `C:\Users\bedna\mktg6620\case3` outside OneDrive, copied assignment files except `labels_test.csv`, and created a Python 3.13 virtual environment. Installed the specified dependencies.
Dataset inspection	Counted products, blank product classes, queries by split, and development/validation labels by grade. Retrieved development query examples and product details.
Script creation	Verified the recipe against `VD3_QUESTIONS.md` and created the commented `embed_and_rank.py`. Presented its code before execution.
Embedding generation	Loaded the two pinned models on CPU, used unchanged product `search_text`, added the required prefix only to BGE queries, and generated normalized float32 vectors.
Ranking generation	Compared each query vector with every product vector using dot products, sorted descending by score with lower numeric product IDs breaking ties, and wrote ten results per query per model.
Diagnostics	Recorded shapes, vector-length ranges, raw bytes, encoding times, versions, and computer information in JSON.
Format validation	Independently checked ranking structure and ran the supplied scorer’s format

checker with the test-label fingerprint check skipped in memory. It reported `Format OK.` |

Dataset counts obtained during Q1:

Item	Count
Products	42,994
Blank or missing product classes, including whitespace-only values	2,852
Development queries	79
Validation queries	200
Test queries, counted from `queries.csv`	200

Label	Development	Validation
Exact	4,623	11,134
Partial	23,222	63,712
Irrelevant	8,736	29,615

Commands executed and recorded outcomes

The principal setup commands were executed through PowerShell. Full paths were used where necessary:

```powershell
py --version
python --version
py -0p

py -3.13 -m venv "C:\Users\bedna\mktg6620\case3\.venv"
```

From the local `case3` directory:

```powershell
& '.\.venv\Scripts\python.exe' -m pip install --upgrade pip
& '.\.venv\Scripts\python.exe' -m pip install -r requirements.txt
& '.\.venv\Scripts\python.exe' -m pip install 'torch==2.11.0' --index-url
https://download.pytorch.org/whl/cpu
& '.\.venv\Scripts\python.exe' -m pip install 'torch==2.11.0' 'sentence-transformers==5.4.1'
'transformers==4.57.6'

& '.\.venv\Scripts\python.exe' -m pip check
```

`pip check` reported:

```text
No broken requirements found.
```

The development baseline was initially attempted with:

```powershell
& '.\.venv\Scripts\python.exe' score.py score --split dev
```

After the in-memory workaround described below, it completed and reported keyword **nDCG@10 = 0.743
across 79 queries**.

The reviewed embedding script was executed with:

```powershell
& '.\.venv\Scripts\python.exe' -u embed_and_rank.py *> embedding_run.log
```

A later rerun, explicitly requested in the session, used:

```powershell
& '.\.venv\Scripts\python.exe' -u embed_and_rank.py > embedding_rerun_stdout.log 2>
embedding_rerun_stderr.log
$embeddingExitCode = $LASTEXITCODE
Write-Output "Embedding Python exit code: $embeddingExitCode"
exit $embeddingExitCode
```

That rerun reported:

```text
Embedding Python exit code: 0
```

Additional commands used `Get-ChildItem`, `Get-Content`, and `Get-CimInstance` for file and hardware inspection. Short Python helpers used `csv`, `Counter`, and installed package metadata for counts, product joins, version checks, and structural checks. They did not access test labels.

Errors, warnings, and corrections

- **Local command startup failure:** Initial sandbox commands failed with `helper_unknown_error: setup refresh had errors`. Subsequent local commands used the approved escalation mechanism.
- **PowerShell quoting error:** The first package-import verification command produced a `SyntaxError` because quotation marks were lost. The quoting was corrected, and the import/version check succeeded.
- **Interrupted script-creation attempt:** An earlier create-and-run command was interrupted. A subsequent inspection found no script or Python process remaining. The script was then created separately for review.
- **Missing test-label fingerprint:** The ordinary development scoring command failed with:

  ```text
  FileNotFoundError: [Errno 2] No such file or directory:
  'C:\\Users\\bedna\\mktg6620\\case3\\data\\labels_test.csv'
  ```

  Although development scoring does not need test judgments, `load_data()` fingerprints every file listed in the manifest. Because the local copy deliberately excluded `labels_test.csv`, startup stopped.

  For the development baseline and later format check, the agent read `score.py` as source text and inserted this condition **in memory only** inside its manifest-check loop:

  ```python
  if name == 'labels_test.csv': continue
  ```

  The adjusted source was executed with the corresponding arguments:

  ```python
  sys.argv = [str(path), 'score', '--split', 'dev']
  ```

  or:

  ```python
  sys.argv = [str(path), 'check', '--rankings', 'my_rankings.csv']
  ```

  Only the test-label fingerprint check was bypassed. Other file checks, development labels, ranking checks, and scoring calculations remained unchanged. Neither `score.py` nor the manifest was edited.

- **Hugging Face warnings:** The first run reported unavailable Windows cache symlinks and fallback from Xet to HTTP downloads. Encoding continued without changes.
- **Docstring warning:** The script contains a Windows command in its opening docstring that triggers `SyntaxWarning: invalid escape sequence '\.'`. Compilation succeeds; this warning does not change the embedding or ranking calculations. The script was left unchanged.
- **First run’s exit status:** The command wrapper reported exit code 1 even though both models completed, the log contained no traceback, and the generated rankings passed structural verification. The

 cause was not established. The later rerun explicitly captured Python’s exit code and returned 0.
- **Timing anomaly:** The later BGE rerun recorded approximately 43.33 hours of elapsed product-encoding time. Sleep or suspension was suggested as a possible cause but was not verified. The earlier run recorded 539.63 seconds for BGE product encoding.

Format-check record

The supplied checker, with only the in-memory fingerprint skip, returned:

```text
bge_small        dev 79/79  validation 200/200  test 200/200
keyword          dev 79/79  validation 200/200  test 200/200
minilm           dev 79/79  validation 200/200  test 200/200
Format OK.
```

This checks ranking coverage and format; it does not score test results. That check occurred before the later rerun. The current file’s row count was reconfirmed today as **9,580**.

AI record: actual prompt excerpts and key responses

These are selected verbatim excerpts from the session, with factual summaries of the corresponding agent responses.

Actual user prompt excerpt	Key agent response or action
“Read VD3_QUESTIONS.md, especially the recipe and Q1 requirements. Do not open, read, preview, or analyze data/labels_test.csv, and do not run test scoring.”	Read the assignment after resolving command-tool access problems; summarized the recipe and Q1 requirements.
“First, inspect the folder structure and tell me whether all required assignment files are present. Check my Python version and operating system.”	Confirmed the supplied filenames, Python 3.13.15, and Windows 11 Home 64-bit. Flagged that the assignment folder was in OneDrive.
“Then help set up a local .venv virtual environment and install the dependencies exactly as specified in the assignment”	Created the local project outside OneDrive, excluded test labels, created `.venv`, and installed the pinned packages.
“report the actual installed versions of Python, torch, sentence-transformers, transformers, and NumPy.”	Reported the verified versions listed above.
“run the development keyword baseline: python score.py score and confirm the keyword nDCG@10 is approximately 0.743”	Encountered the missing-file fingerprint error, disclosed the in-memory workaround, and obtained 0.743 on 79 development queries.
“Read the CSV files and report: total number of products, number of queries in each split”	Counted records directly from the permitted CSV files and reported product, query, and development/validation label totals.
“Count how many rows in data/products.csv have a blank or missing product_class. Report the exact count.”	Reported 2,852, treating whitespace-only fields as blank.
“pick one development query, quote one product labelled extract and one labelled irrevant and say in a sentence why each label fits”	Retrieved query 421, “dining room tables,” with labelled products 214 and 2726; also supplied brief label explanations.
“provide me another development query, quote one product labelled exact and one irrelevant.”	Retrieved query 434, “bathroom lighting,” with Exact product 210 and Irrelevant product 5.
“Create the script but DO NOT execute it.”	Verified the recipe, created `embed_and_rank.py`, and displayed the full code without executing it.
“I have reviewed embed_and_rank.py, and the implementation appears to match the assignment's recipe.”	Executed the existing script after authorization and reported measured diagnostics and rankings.
“Do not interpret whether the products make sense. I will do that myself.”	Retrieved the two top-five lists for query 1 and reported factual results without judging their suitability.
“please run the python embed_and_rank.py and record the following for each model”	Performed the explicitly requested rerun, reported exit code 0, and disclosed the anomalous BGE timing.
“Do not rerun the embedding models.”	For this request, read existing outputs, verified installed package metadata, and checked script syntax without executing the models.

The statement that you reviewed the script comes directly from your prompt. This record does not claim any additional checks or assessments on your behalf.

Final code availability

The complete, **155-line** final script is available at [embed_and_rank.py](C:/Users/bedna/mktg6620/case3/embed_and_rank.py) for inclusion under the **Code** section of `answers.md`. Its syntax was checked without executing it; the docstring warning noted above remains.

`answers.md` was not modified. No test labels were inspected, no validation or test scoring ran, and no Q2 work was performed for this request.

072 | Turn 18 | USER PROMPT

2026-10-22T01:28:41.973Z

```
We are beginning Q2a of MKTG 6620 Case 3.

The embedding script has already been executed, and my_rankings.csv has passed the structural format
check. Do not rerun embeddings.

Before scoring, check whether data/labels_test.csv exists. If it is missing, restore the original file
from VD3_student_files.zip without displaying, parsing, or analyzing its contents. Do not modify score.py
 or data_manifest.json. The supplied scorer may perform its normal automatic fingerprint verification.

Then run:

python score.py check --rankings my_rankings.csv

python score.py score --split validation --rankings my_rankings.csv

Read outputs/validation_metrics.csv and report the following for keyword, minilm, and bge_small:

- n
- nDCG@10
- Exact@1
- Exact@1 95% interval (exact1_low through exact1_high)
- judged@10

For minilm and bge_small, also report:

- nDCG@10 difference versus keyword
- Paired 95% interval (diff_low through diff_high)
- beats_baseline
- loses_to_baseline

Present all results in one Markdown table using the exact column structure requested in Q2a.

Verify whether:
- MiniLM nDCG@10 is within 0.003 of 0.736.
- BGE-small nDCG@10 is within 0.003 of 0.739.

If either differs, examine the recipe settings and identify the possible discrepancy. Do not change the
embedding script or rankings without discussing the problem first.

IMPORTANT:
- Do not open, display, or analyze test labels.
- Do not run test scoring.
- Do not create selection.json.
- Do not choose a model for me.
- Do not modify answers.md.

Stop after presenting the validation table and checks.
```

073 | Turn 18 | ASSISTANT UPDATE

2026-10-22T01:28:46.203Z

I'll check whether the local test-label file exists and locate the original ZIP if it needs restoring. Then I'll run the unmodified scorer's format check and validation scoring, report the Q2a table, and

compare both embedding scores with the recipe targets.

074 | Turn 18 | ASSISTANT REPLY

2026-10-22T01:29:51.025Z
The missing `data/labels_test.csv` was restored from the original ZIP without displaying or parsing its contents. The unmodified scorer’s normal fingerprint verification succeeded, and the rankings check reported **`Format OK.`**

Results from `outputs/validation_metrics.csv`:

Method	n	nDCG@10	Exact@1 (95% interval)	judged@10	nDCG@10 difference vs. keyword (paired 95% interval)	Queries better / worse than keyword
keyword	200	0.6750	0.4500 (0.3826–0.5192)	0.8905	—	—
minilm	200	0.7357	0.4800 (0.4118–0.5490)	0.8780	+0.0607 (+0.0241–+0.0968)	91 / 58
bge_small	200	0.7390	0.5200 (0.4510–0.5882)	0.8805	+0.0641 (+0.0278–+0.1002)	90 / 61

Both match the recipe targets:

- **MiniLM:** |0.7357 – 0.736| = **0.0003**, within 0.003.
- **BGE-small:** |0.7390 – 0.739| = **0.0000**, within 0.003.

No discrepancy investigation was needed. Embeddings were not rerun; no scripts, rankings, manifest, or `answers.md` were modified. No test scoring ran, and no `selection.json` was created.

075 | Turn 19 | USER PROMPT

2026-10-22T01:37:50.392Z
Use validation query 0 and run:

```
python score.py show --query-id 0 --rankings my_rankings.csv
```

Do not access test labels or test scores.

For both MiniLM and BGE-small, report:

The query text.

The ten ranked products and their product IDs.

Each product's relevance label (Exact, Partial, Irrelevant, or Unjudged).

The corresponding numerical gain (Exact = 1, Partial = 0.5, Irrelevant or Unjudged = 0).

Also report:

The total number of Exact and Partial products labelled for this query, as shown by the scorer.

The query's nDCG@10 for each model from outputs/validation_per_query.csv.

Present the information using simple plain-text formatting, not elaborate Markdown tables.

Do not calculate DCG, ideal DCG, or nDCG for me. I need to work through those calculations myself.

Do not select a model, create selection.json, run test scoring, or modify answers.md.

Stop after presenting the requested information.

076 | Turn 19 | ASSISTANT UPDATE

2026-10-22T01:37:55.151Z
I’ll retrieve query 0’s rankings, labels, and existing per-query scores. Since the unmodified `show` command fingerprints test labels at startup, I’ll use the earlier in-memory skip for that fingerprint

only. I won’t calculate DCG, ideal DCG, or nDCG.

077 | Turn 19 | ASSISTANT REPLY

2026-10-22T01:38:26.825Z
Validation query 0: “salon chair”

The scorer reports 3,302 labelled products for this query, including:
Exact: 39
Partial: 2,977

MiniLM (minilm)
Existing nDCG@10 from outputs/validation_per_query.csv: 0.500000

```text
Rank | Product ID | Product name                                        | Label   | Gain
1    | 9234       | beauty salon task chair                             | Partial | 0.5
2    | 24010      | bar salon task chair                                | Partial | 0.5
3    | 18273      | fashion casual office salon task chair              | Partial | 0.5
4    | 18276      | evelinde fashion casual office salon task chair     | Partial | 0.5
5    | 18272      | fashion casual beauty salon task chair              | Partial | 0.5
6    | 18277      | eveline fashion casual office salon task chair      | Partial | 0.5
7    | 19456      | fashion beauty salon conference chair               | Partial | 0.5
8    | 42329      | fashion casual beauty salon office chair            | Partial | 0.5
9    | 18274      | evelijne fashion casual office beauty salon task chair | Partial | 0.5
10   | 18275      | evelinda fashion casual office salon task chair     | Partial | 0.5
```

BGE-small (bge_small)
Existing nDCG@10 from outputs/validation_per_query.csv: 0.756764

```text
Rank | Product ID | Product name                                    | Label   | Gain
1    | 7465       | hair salon chair                                | Exact   | 1
2    | 36910      | beauty spa salon barber chair                   | Exact   | 1
3    | 9234       | beauty salon task chair                         | Partial | 0.5
4    | 18272      | fashion casual beauty salon task chair          | Partial | 0.5
5    | 25431      | barberpub salon massage chair                   | Exact   | 1
6    | 19456      | fashion beauty salon conference chair           | Partial | 0.5
7    | 42329      | fashion casual beauty salon office chair        | Partial | 0.5
8    | 39428      | barber salon reclining massage chair            | Exact   | 1
9    | 27443      | beauty salon ergonomic task chair               | Partial | 0.5
10   | 40996      | fashion casual work beauty salon task chair     | Partial | 0.5
```

The show command used the in-memory test-label fingerprint skip; score.py was unchanged. No test labels or test scores were accessed, no selection was created, and answers.md was not modified.

078 | Turn 20 | USER PROMPT

2026-10-22T02:12:06.278Z
BGE-Small is my choice for my encoder. Even though it has a longer encoding time, it has the highest validation for nDCG@10 and Exact@1.

079 | Turn 20 | ASSISTANT REPLY

2026-10-22T02:12:10.449Z
Your chosen encoder is **BGE-small (`bge_small`)**. The validation results support your stated reason:

- nDCG@10: **0.7390**, the highest of the three methods.
- Exact@1: **0.5200**, also the highest.

No `selection.json` has been created yet.

080 | Turn 21 | USER PROMPT

2026-10-22T02:12:33.682Z
python score.py select --method bge_small --reason "Although it has a longer encoding time, BGE-small achieved the highest validation nDCG@10 (0.7390) and Exact@1 (0.5200) of the three methods."

081 | Turn 21 | ASSISTANT UPDATE

2026-10-22T02:12:37.368Z
I'll save your BGE-small selection with your exact reason. I won't run test scoring.

082 | Turn 21 | ASSISTANT REPLY

2026-10-22T02:12:50.984Z
Saved [selection.json](C:/Users/bedna/mktg6620/case3/selection.json):

```json
{
  "method": "bge_small",
  "reason": "Although it has a longer encoding time, BGE-small achieved the highest validation nDCG@10 (0.7390) and Exact@1 (0.5200) of the three methods.",
  "saved_at_utc": "2026-10-22T02:12:44Z",
  "validation_metrics_sha256": "2feee98656c3d9a9cbfff4935951a1bd0fde0d58001ba971900a373d2443388f"
}
```

No test scoring was run.

083 | Turn 22 | USER PROMPT

2026-10-22T02:13:36.347Z
Run the test exactly once

If the selection is valid and the test has not already been scored, run:

python score.py score --split test --selection selection.json --rankings my_rankings.csv

Use the original, unmodified score.py.

Do not change the rankings, model settings, or selection.

STEP 3 — Extract test results

Read outputs/test_metrics.csv and report the following for keyword and bge_small:

n (number of test queries)

nDCG@10

Exact@1

Wilson 95% interval for Exact@1 (exact1_low to exact1_high)

judged@10

nDCG@10 difference versus keyword

Paired 95% interval for the nDCG difference (diff_low to diff_high)

Number of queries better than keyword (beats_baseline)

Number of queries worse than keyword (loses_to_baseline)

Present the results in simple, clearly labelled plain text rather than an elaborate Markdown table.

STEP 4 — Compare validation and test

Retrieve the corresponding keyword and BGE-small metrics from outputs/validation_metrics.csv.

Show the validation and test nDCG@10 values and paired differences side by side.

Also report:

Whether the test difference is positive or negative.

Whether the test paired 95% interval includes zero.

Whether BGE-small performed better or worse on test than on validation.

Any differences in Exact@1 and judged@10.

084 | Turn 22 | ASSISTANT UPDATE

2026-10-22T02:13:42.491Z
I'll verify the selection, confirm `score.py` matches the supplied original, and check for existing test outputs or logged test runs. If those checks pass and the test hasn't been scored, I'll run it once and report the test results alongside validation.

085 | Turn 22 | ASSISTANT REPLY

2026-10-22T02:15:16.217Z
The test was scored **exactly once**. Before running it, I confirmed that the selection was valid, `score.py` matched the original ZIP byte-for-byte, and no prior test outputs or logged test runs were found. Rankings, settings, and selection were unchanged.

Test results — keyword
```text
n:                              200
nDCG@10:                        0.6846
Exact@1:                        0.4250
Exact@1 Wilson 95% interval:     0.3585 to 0.4943
judged@10:                      0.8505
nDCG@10 difference vs keyword:   0.0000
Paired 95% interval:            0.0000 to 0.0000
Queries better than keyword:    0
Queries worse than keyword:     0
```

Test results — bge_small
```text
n:                              200
nDCG@10:                        0.7100
Exact@1:                        0.4600
Exact@1 Wilson 95% interval:     0.3923 to 0.5292
judged@10:                      0.8315
nDCG@10 difference vs keyword:  +0.0254
Paired 95% interval:           -0.0022 to +0.0527
Queries better than keyword:    81
Queries worse than keyword:     70
```

Validation and test comparison
```text
Measure                         Validation       Test
Keyword nDCG@10                 0.6750           0.6846
BGE-small nDCG@10               0.7390           0.7100
BGE-small difference            +0.0641          +0.0254
Paired 95% interval             +0.0278–+0.1002   -0.0022–+0.0527

Keyword Exact@1                 0.4500           0.4250
```

```
BGE-small Exact@1               0.5200          0.4600
Keyword judged@10               0.8905          0.8505
BGE-small judged@10              0.8805          0.8315
```

BGE-small's test difference is **positive**, but its paired 95% interval **includes zero**. The test results therefore remain compatible with no average nDCG advantage.

BGE-small performed worse on test than validation: nDCG@10 decreased by **0.0290**, Exact@1 decreased by **6 percentage points**, and judged@10 decreased by **4.9 percentage points**. Its advantage over keyword narrowed from **0.0641** to **0.0254**.

Keyword's Exact@1 decreased by **2.5 percentage points**, and judged@10 decreased by **4 percentage points**.

086 | Turn 23 | USER PROMPT

2026-10-23T00:48:26.075Z

My selected method is bge_small. We have completed validation scoring and the one-time test. For Q3, use VALIDATION data only.

Do not rerun embeddings or test scoring. Do not modify selection.json, my_rankings.csv, score.py, or answers.md.

Step 1 — Identify the six queries

Read outputs/validation_per_query.csv and identify:

The three validation queries where bge_small beat keyword by the largest positive nDCG@10 difference.

The three validation queries where keyword beat bge_small by the largest negative nDCG@10 difference.

For each query, report:

Query ID

Exact query text

Keyword nDCG@10

BGE-small nDCG@10

Difference (BGE-small minus keyword)

Use the actual per-query scores and sort by the differences. Do not invent or estimate any values.

Step 2 — Inspect each query

For each of the six query IDs, run:

python score.py show --query-id QUERY_ID --rankings my_rankings.csv

Replace QUERY_ID with the actual query ID.

For each query, report:

The ten ranked products from keyword.

The ten ranked products from bge_small.

Product IDs and names.

Relevance labels (Exact, Partial, Irrelevant, or Unjudged).

The number of Exact and Partial labelled products for that query.

Step 3 — Organize the results

Present the six queries individually, with simple plain-text formatting rather than elaborate Markdown tables.

Clearly separate:

Three BGE-small wins.

Three keyword wins.

Highlight factual differences in the rankings, such as where one method retrieved Exact products and another returned Partial or Irrelevant products.

Do not write my explanations of what went right or wrong.
Do not classify the failures or assign failure types.
Do not make my personal judgments about the relevance labels.

I will review the six results, name the important products, and write my own analysis for Q3a.

Stop after reporting these six queries. Do not begin Q3b or Q4.

087 | Turn 23 | ASSISTANT UPDATE

2026-10-23T00:48:31.014Z
I'll sort the validation per-query scores to identify the six largest differences, then inspect those queries' rankings and labels. I'll skip test-label fingerprinting in memory so only validation data is accessed, and leave all protected files unchanged.

088 | Turn 23 | ASSISTANT UPDATE

2026-10-23T00:49:57.833Z
The largest BGE-small wins are queries 168, 345, and 68; the largest keyword wins are 219, 360, and 416. I've retrieved all six rankings. A few accented characters and symbols were garbled in the Windows output, so I'll verify those names before presenting them.

089 | Turn 23 | ASSISTANT REPLY

2026-10-23T01:15:32.985Z
These are the six largest differences, sorted using the scores stored in `outputs/validation_per_query.csv`. Difference means BGE-small minus keyword. Ties, if any, were ordered by query ID.

Each query's `show` command ran with only test-label fingerprinting skipped in memory. No protected files were modified, and no test data or scores were accessed.

THREE BGE-SMALL WINS

Query 168 — "printers"
```text
Keyword nDCG@10:    0.000000
BGE-small nDCG@10:  1.000000
Difference:       +1.000000

Labelled products: 68 total; 0 Exact; 48 Partial

KEYWORD
Rank | Product ID | Product name | Label
1  | 0 | solid wood platform bed | Unjudged
2  | 1 | all-clad 7 qt . slow cooker | Unjudged
3  | 2 | all-clad electrics 6.5 qt . slow cooker | Unjudged
4  | 3 | all-clad all professional tools pizza cutter | Unjudged
5  | 4 | baldwin prestige alcott passage knob with round rosette | Unjudged
6  | 5 | vogan 33 '' single bathroom vanity | Unjudged
```

```
7  | 6 | vogelsang 48 '' single bathroom vanity | Unjudged
8  | 7 | 36 '' single bathroom vanity | Unjudged
9  | 8 | erith obliqui urn | Unjudged
10 | 9 | vezina 65 '' rolled arm chesterfield loveseat | Unjudged

BGE-SMALL
Rank | Product ID | Product name | Label
1  | 2295 | printer stand | Partial
2  | 26471 | printer stand | Partial
3  | 21992 | home office printer stand | Partial
4  | 974 | office printer stand | Partial
5  | 39378 | printer stand book cart | Partial
6  | 34380 | home office mobile printer stand | Partial
7  | 27625 | printer stand with wheels | Partial
8  | 37869 | mobile printer stand with casters | Partial
9  | 15410 | mobile printer stand | Partial
10 | 26469 | classify mobile printer stand | Partial
```

Factual difference: BGE-small returned ten Partial products; keyword returned ten Unjudged products. This query has no Exact-labelled products.

Query 345 — “oriental vanity”

```text
Keyword nDCG@10:    0.000000
BGE-small nDCG@10:  0.926636
Difference:       +0.926636

Labelled products: 1,148 total; 0 Exact; 757 Partial

KEYWORD
Rank | Product ID | Product name | Label
1  | 30052 | oriental oriental red/gray area rug | Irrelevant
2  | 7219 | ottis oriental oriental black/tan area rug | Irrelevant
3  | 26890 | nerbone oriental power loom red oriental rug | Irrelevant
4  | 2618 | clarence oriental beige rug | Unjudged
5  | 4090 | bonney oriental area rug | Irrelevant
6  | 6190 | oriental teal indoor/outdoor area rug | Irrelevant
7  | 23710 | hosking oriental blue rug | Irrelevant
8  | 26897 | brandt oriental brown rug | Irrelevant
9  | 26900 | brandt oriental beige rug | Irrelevant
10 | 26908 | brandt oriental blue rug | Irrelevant

BGE-SMALL
Rank | Product ID | Product name | Label
1  | 19503 | ilani vanity set with stool and mirror | Partial
2  | 9195 | vanity | Partial
3  | 8631 | choi vanity | Partial
4  | 42028 | froh vanity | Partial
5  | 42029 | frohlinde vanity | Partial
6  | 19505 | gab vanity set with stool and mirror | Partial
7  | 33756 | oriental 6 '' x 12 '' marble marble look tile | Irrelevant
8  | 42026 | frodine vanity | Partial
9  | 4582 | amishi vanity | Partial
10 | 7117 | cornelison dressing vanity set with stool and mirror | Partial
```

Factual difference: BGE-small returned nine Partial products and one Irrelevant product. Keyword returned nine Irrelevant products and one Unjudged product.

Query 68 — “nautical platters”

```text
Keyword nDCG@10:    0.110046
BGE-small nDCG@10:  0.952606
Difference:       +0.842560
```

Labelled products: 204 total; 89 Exact; 79 Partial

KEYWORD
Rank | Product ID | Product name | Label
1 | 39599 | wilton armetale nautical long bread tray | Partial
2 | 11816 | nautical upholstered panel headboard | Irrelevant
3 | 31872 | nautical and coastal blue rustic anchor nautical watercrafts - graphic art print | Irrelevant
4 | 18414 | nautical anchor wall décor | Irrelevant
5 | 3517 | nautical wood anchor wall décor | Irrelevant
6 | 13029 | nautical anchor compass wall décor | Irrelevant
7 | 30398 | nautical reclining chaise lounge | Irrelevant
8 | 30401 | nautical reclining chaise lounge | Irrelevant
9 | 40760 | nautical ship wheel wall décor | Irrelevant
10 | 10314 | love anchor nautical duvet cover set | Irrelevant

BGE-SMALL
Rank | Product ID | Product name | Label
1 | 13959 | boat shaped platter | Exact
2 | 16388 | coastal platter | Exact
3 | 49 | hiltner seashell platter | Exact
4 | 2425 | coastal fish platter | Partial
5 | 4612 | haislip boat platter | Exact
6 | 16390 | coastal medium platter | Exact
7 | 2431 | coastal crab platter | Exact
8 | 41410 | sailboat tray and small bowl chip and dip platter | Exact
9 | 28339 | seaside platter | Exact
10 | 26667 | seaside large mussel platter | Exact
```

Factual difference: BGE-small returned nine Exact products and one Partial product. Keyword returned one Partial product and nine Irrelevant products.

THREE KEYWORD WINS

Query 219 — “trinaic towel rod”
```text
Keyword nDCG@10:    1.000000
BGE-small nDCG@10:  0.069431
Difference:       -0.930569

Labelled products: 38 total; 0 Exact; 38 Partial

KEYWORD
Rank | Product ID | Product name | Label
1 | 2926 | single rod and multi-rod 25 '' fixture mounted towel bar | Partial
2 | 2927 | single rod and multi-rod 24 '' fixture mounted towel bar | Partial
3 | 2924 | single rod wall mounted towel rack | Partial
4 | 2925 | multi-rod wall mounted towel rack | Partial
5 | 4804 | stainless steel towel rack punch-free towel single rod rack bath towel holder | Partial
6 | 4805 | stainless steel painted towel rack punch-free towel single rod rack bath towel holder | Partial
7 | 10677 | 16 inch black single towel bar rack , sus 304 stainless steel bathroom towel bar heavy duty towel holder kitchen towel rod , wall mount with screws towel bar rod matte black finish | Partial
8 | 1667 | 24 '' wall mounted single rod towel rack | Partial
9 | 1669 | 24 '' wall mounted multi rod towel rack | Partial
10 | 10679 | bathroom double towel bar towel hanging rod/rail towel rack holder storage wall mount 24-inch,304 stainless steel matte black | Partial

BGE-SMALL
Rank | Product ID | Product name | Label
1 | 10114 | trinsic® wall mounted robe hook | Unjudged
2 | 10115 | trinsic® bathroom towel ring | Unjudged
3 | 27666 | triva wall mounted robe hook | Unjudged
4 | 10183 | trinsic 24 '' wall mounted towel bar | Unjudged
5 | 10180 | trinsic® 30 '' wall mounted towel bar | Unjudged

6  | 26953 | peterson adjustable drapery tringle single curtain rod | Unjudged
7  | 6598 | trinsic 30 '' wall mounted towel bar | Unjudged
8  | 2926 | single rod and multi-rod 25 '' fixture mounted towel bar | Partial
9  | 6599 | trinsic 18 '' wall mounted towel bar | Unjudged
10 | 10138 | trinsic 18 '' wall mounted towel bar | Unjudged
```

Factual difference: Keyword returned ten Partial products. BGE-small returned one Partial product at rank 8 and nine Unjudged products. Product 2926 appears at keyword rank 1 and BGE-small rank 8.

Query 360 — “huntsville sectional sofa”
```text
Keyword nDCG@10:    1.000000
BGE-small nDCG@10:  0.305235
Difference:       -0.694765

Labelled products: 125 total; 0 Exact; 125 Partial

KEYWORD
Rank | Product ID | Product name | Label
1  | 537 | sectional sofa in gray linen | Partial
2  | 9687 | kendall sectional sofa with ottoman | Partial
3  | 33593 | sectional couch with reversible chaise modern l-shape sofa 3-seat couch modular sectional sofa | Partial
4  | 23611 | nina left facing sectional sofa . taupe . | Partial
5  | 23612 | allisa right facing sectional sofa . grey . | Partial
6  | 33595 | convertible modular sectional sofa u shaped couch with reversible chaise modular couch sectional sofa with ottomans | Partial
7  | 9686 | hayllie woven reversible sectional sofa with ottoman | Partial
8  | 20537 | paille outdoor sofa patio sectional with cushions | Partial
9  | 8243 | 115 '' wide left hand facing reclining corner sectional sofa | Partial
10 | 33592 | 6-piece patio furniture set corner sofa set pe rattan wicker outdoor garden sectional sofa chair | Partial

BGE-SMALL
Rank | Product ID | Product name | Label
1  | 1875 | russ 103.5 '' wide linen sectional with ottoman | Partial
2  | 6284 | sectional with cushions | Unjudged
3  | 5647 | alp patio sectional with cushions | Unjudged
4  | 25074 | caspian patio sectional with cushion | Unjudged
5  | 9687 | kendall sectional sofa with ottoman | Partial
6  | 29792 | lucaya patio sectional with cushions | Unjudged
7  | 26620 | albrah patio sectional with cushions | Unjudged
8  | 15277 | patio sectional with cushions | Unjudged
9  | 12439 | patio sectional with cushions | Unjudged
10 | 25091 | almine patio sectional with cushions | Unjudged
```

Factual difference: Keyword returned ten Partial products. BGE-small returned two Partial products and eight Unjudged products. Product 9687 appears at keyword rank 2 and BGE-small rank 5.

Query 416 — “wayfair coffee table”
```text
Keyword nDCG@10:    1.000000
BGE-small nDCG@10:  0.416567
Difference:       -0.583433

Labelled products: 1,447 total; 0 Exact; 1,443 Partial

KEYWORD
Rank | Product ID | Product name | Label
1  | 608 | coffee table | Partial
2  | 20513 | coffee table | Partial
3  | 22679 | coffee table | Partial
4  | 33290 | coffee table | Partial
5  | 19750 | a-37 coffee table in , matching coffee table | Partial

```
6  | 290 | kalmus coffee table | Partial
7  | 294 | aalst coffee table | Partial
8  | 295 | buchheit coffee table | Partial
9  | 296 | aalt coffee table | Partial
10 | 297 | aaltien coffee table | Partial

BGE-SMALL
Rank | Product ID | Product name | Label
1  | 39189 | wayfair basics® tablecloth | Unjudged
2  | 36109 | raffa coffee table | Partial
3  | 39160 | wayfair basics stackable storage drawers | Unjudged
4  | 39169 | wayfair basics solid color round tablecloth | Unjudged
5  | 39168 | wayfair basics solid color square tablecloth | Unjudged
6  | 17106 | telfair sled coffee table with storage | Partial
7  | 39213 | wayfair basics® industrial stool | Unjudged
8  | 38258 | heflin coffee table | Partial
9  | 32149 | tillar coffee table | Partial
10 | 34533 | huyet coffee table | Partial
```

Factual difference: Keyword returned ten Partial products. BGE-small returned five Partial products and five Unjudged products, including the Unjudged tablecloth at rank 1.

090 | Turn 24 | USER PROMPT

2026-10-23T01:42:50.076Z
My selected model is bge_small. Use validation data only.

Do not rerun embeddings, validation scoring, or test scoring. Do not modify any existing files.

First, retrieve judged@10 from outputs/validation_metrics.csv for:

Keyword

BGE-small

Next, identify ten Unjudged products appearing in BGE-small's top-ten validation rankings. Use ten different queries if possible.

For each product, retrieve:

Query ID

Exact query text

Product ID

Product name

Product class

The beginning of product_description (approximately the first 150 characters)

Product's rank in BGE-small's top ten

Confirm that each selected query belongs to the validation split and that each product has no relevance label for that query.

Present the results in simple plain text, numbered 1 through 10. Do not create a large Markdown table.

Do not assign Exact, Partial, or Irrelevant labels. Do not evaluate whether the products are relevant. I will make those judgments myself.

Do not write my Q3b interpretation, begin Q3c, or edit answers.md.

Stop after reporting the ten products and judged@10 values.

091 | Turn 24 | ASSISTANT UPDATE

2026-10-23T01:42:55.373Z
I'll read the saved validation metrics and find ten unjudged BGE-small results from ten different validation queries. I'll confirm that each product has no label for its query and report the product details without judging relevance.

092 | Turn 24 | ASSISTANT REPLY

2026-10-23T01:43:38.368Z
Validation judged@10:
Keyword: 0.8905
BGE-small: 0.8805

The ten examples below come from ten different queries. For every example, I confirmed that the query's split is validation and that no matching query–product pair appears in `labels_validation.csv`. Examples were selected by ascending query ID, taking the first unjudged BGE-small result for each query.

1. Query ID: 4
   Query: "chair and a half recliner"
   Product ID: 33723
   Product name: side chair
   Product class: Dining Chairs
   BGE-small rank: 1
   Description beginning: "the sole designs dining chairs is a simple yet attractive option to use in any room of your home . featuring a contemporary style , this side chair is"

2. Query ID: 23
   Query: "stoneford end tables white and wood"
   Product ID: 3078
   Product name: saltford twisted teak side table
   Product class: End Tables
   BGE-small rank: 1
   Description beginning: Blank in the source CSV.

3. Query ID: 29
   Query: "bathroom vanity knobs"
   Product ID: 20628
   Product name: bathroom/vanity mirror
   Product class: Wall & Accent Mirrors
   BGE-small rank: 1
   Description beginning: "evoking a distressed barn wood aesthetic , this mirror shines in coastal or farmhouse ensembles ! crafted of solid and manufactured wood , the rectang"

4. Query ID: 34
   Query: "bar stool with backrest"
   Product ID: 1608
   Product name: stool
   Product class: Bar Stools
   BGE-small rank: 3
   Description beginning: "it is finished with powder coating and can be used indoors and outdoors . undamaged feet provide floor protection and can be stacked for efficient sto"

5. Query ID: 35
   Query: "enclosed shoe rack"
   Product ID: 8412
   Product name: entryway shoe storage bench
   Product class: Benches|Shoe Storage
   BGE-small rank: 7
   Description beginning: "this neat and compact shoe bench will help organize your shoes , heels , slippers , sandals , and any other shoe-related supply while providing a seat"

6. Query ID: 42

   Query: “toddler couch fold out”
   Product ID: 10429
   Product name: convertible toddler bed
   Product class: Kids Beds
   BGE-small rank: 1
   Description beginning: Blank in the source CSV.

7. Query ID: 46
   Query: “beach blue headboard”
   Product ID: 16614
   Product name: vay bay bookcase headboard
   Product class: Headboards
   BGE-small rank: 6
   Description beginning: “take a stance for a clean-lined design with the bookcase headboard . rich on the rustic character and decidedly simple on the details , this headboard”

8. Query ID: 52
   Query: “westling coffee table”
   Product ID: 38561
   Product name: westling traditional standard bed
   Product class: Beds
   BGE-small rank: 2
   Description beginning: “this handcrafted bed is made from responsibly-sourced northern white cedar , shipped to you ready for you to assemble . for the true do-it-yourselfer ”

9. Query ID: 57
   Query: “jordanna solid wood rocking”
   Product ID: 5378
   Product name: kiana solid wooden end table
   Product class: End Tables
   BGE-small rank: 3
   Description beginning: “elegantly beveled with corners this solid wood end table is simple yet very eye-catchy furniture , with its clean and crisp finish . made of solid woo”

10. Query ID: 60
    Query: “capricorn chest”
    Product ID: 36085
    Product name: capricorni 71.6 '' velvet square arm sleeper
    Product class: Blank in the source CSV.
    BGE-small rank: 9
    Description beginning: “make any space feel like home with the love seat sofa bed . classically styled with modern square arms and tufted back , this two-person sofa adds a c”

093 | Turn 25 | USER PROMPT

2026-10-24T00:48:01.952Z
My selected model from Q2 is bge_small. I want to test one improvement using a hybrid search that combines BGE-small and keyword search through Reciprocal Rank Fusion (RRF).

Please follow these steps:

Read the Q4a requirements in VD3_QUESTIONS.md and inspect the existing rankings files.

Create a new Python script named hybrid_rank.py that combines the existing keyword rankings from data/keyword_rankings.csv with BGE-small rankings from my_rankings.csv.

For each product, calculate its combined RRF score by adding 1 / (60 + rank) from each ranking in which it appears.

Sort products by descending combined RRF score. Break ties using the lower numeric product_id. Keep the top 10 products per query.

Use the method name hybrid_rrf and save the rankings in my_change.csv with the required columns: query_id, method, rank, product_id

Generate rankings for all 479 queries so that the output follows the scorer's format. Do not access relevance labels or test scores to build the hybrid method.

Score the new method on VALIDATION ONLY, comparing it against bge_small:

python score.py score --split validation --rankings my_rankings.csv my_change.csv --baseline bge_small --out outputs_q4

Report:

BGE-small validation nDCG@10

Hybrid validation nDCG@10

Hybrid judged@10

Difference in nDCG@10 versus BGE-small

Paired 95% confidence interval

Whether the interval includes zero

Number of queries where the hybrid performs better or worse

Explain factually how the rankings were combined and report the additional encoding time and storage requirements. Distinguish raw vector storage from the storage required for ranking lists.

Do not:

Rerun embeddings.

Change the selected model or selection.json.

Score the test set again.

Modify my_rankings.csv or score.py.

Edit answers.md.

Write my personal interpretation.

If an error occurs, identify it and make only the necessary correction.

Present the results in simple plain text, without elaborate tables.

094 | Turn 25 | ASSISTANT UPDATE

2026-10-24T00:48:06.203Z
I’ll read Q4a and inspect the ranking files, then create the RRF script using only the two existing ranking lists. After generating `my_change.csv`, I’ll score validation against BGE-small and report the metrics, runtime, and added storage.

095 | Turn 25 | ASSISTANT UPDATE

2026-10-24T00:49:01.391Z
The hybrid rankings are generated: 4,790 rows for all 479 queries, with no labels or query text used to build them. Fusion took 0.226 seconds and required no new embeddings. I’ll now run the requested validation comparison against BGE-small.

096 | Turn 25 | ASSISTANT REPLY

2026-10-24T00:49:28.555Z
Created [hybrid_rank.py](C:/Users/bedna/mktg6620/case3/hybrid_rank.py) and generated `my_change.csv` with 4,790 rows: ten results for each of all 479 queries.

For each query, the script combined the existing keyword and BGE-small top-ten lists. Each appearance contributed `1 / (60 + rank)`; products appearing in both lists received both contributions. Products were sorted by descending combined score, with lower numeric product IDs breaking ties. The top ten were saved as `hybrid_rrf`. No labels or query text were used to build the rankings.

Validation results, comparing against BGE-small:

```text
Number of validation queries:       200
BGE-small nDCG@10:                   0.7390
Hybrid nDCG@10:                      0.7182
Hybrid judged@10:                    0.8935
Hybrid nDCG@10 difference:          -0.0209
Paired 95% confidence interval:     -0.0392 to -0.0034
Interval includes zero:             No
Queries hybrid beat BGE-small:       61
Queries hybrid lost to BGE-small:    83
Queries tied:                       56
```

The difference is the scorer’s paired result, computed before rounding the displayed method averages.

Additional runtime and storage:

```text
Additional encoding time:              0 seconds
Fusion time, including reading/writing: 0.226 seconds
Additional raw vector storage:         0 bytes
New hybrid ranking CSV:                107,976 bytes
```

The hybrid reuses existing rankings and creates no vectors. BGE-small’s raw product vectors require 66,038,784 bytes; that requirement is unchanged. Ranking-list storage is separate: the existing keyword CSV is 93,339 bytes, and `my_rankings.csv`, containing both embedding methods, is 202,103 bytes. The new script and diagnostics JSON are additional small files.

Results are saved in [outputs_q4/validation_metrics.csv](C:/Users/bedna/mktg6620/case3/outputs_q4/validation_metrics.csv).

No errors occurred. Embeddings and test scoring were not rerun. `selection.json`, `my_rankings.csv`, `score.py`, and `answers.md` were unchanged.

097 | Turn 26 | USER PROMPT

2026-10-24T01:00:47.961Z
My selected model is bge_small. I tested hybrid_rrf in Q4a, but it performed worse, so use BGE-small for this production estimate.

BRG Direct is assumed to have 2,000,000 products.

Use my actual measurements from embedding_diagnostics.json:

42,994 products

384-dimensional float32 vectors

66,038,784 raw product-vector bytes

BGE-small product encoding time: 539.63 seconds

Computer: Surface Laptop 4, AMD Ryzen 5, four CPU threads

Calculate:

Raw vector storage required for 2,000,000 products, in bytes, MB, and GiB.

Estimated time to embed 2,000,000 products on my computer, assuming encoding time scales linearly.

Convert the estimated time into hours and minutes.

Explain the calculation formulas so I can verify the results.

Then provide brief factual implementation considerations for:

When new products should be embedded.

When existing products should be re-embedded.

What happens when the embedding model changes.

One quality check before launching a new search index.

Metrics that could be monitored after launch.

Use simple plain text, not a Markdown table.

Do not write my personal business recommendations or interpret the production feasibility for me.

Do not run embeddings, access test data, change any files, or modify answers.md.

Stop after providing the calculations and factual implementation considerations.

098 | Turn 26 | ASSISTANT UPDATE

2026-10-24T01:00:52.186Z
I'll calculate storage from the vector dimensions and estimate encoding time using your specified 539.63-second measurement, then list brief implementation considerations. I won't access data or change files.

099 | Turn 26 | ASSISTANT REPLY

2026-10-24T01:01:03.220Z
Raw vector storage

Each product requires:
384 dimensions × 4 bytes per float32 = 1,536 bytes.

For 2,000,000 products:
2,000,000 × 1,536 = 3,072,000,000 bytes.

Conversions:
- MB: 3,072,000,000 ÷ 1,000,000 = 3,072 MB.
- GiB: 3,072,000,000 ÷ 1,073,741,824 ≈ 2.861 GiB.

This covers raw product vectors only. Product metadata, index structures, ranking lists, and backups require additional storage.

Estimated encoding time

Using your specified measurement of 539.63 seconds for 42,994 products:

Scale factor:
2,000,000 ÷ 42,994 ≈ 46.5181.

Estimated time:
539.63 × 46.5181 ≈ 25,102.57 seconds.

Conversions:
- 418.38 minutes.
- 6.973 hours.

- Approximately 6 hours, 58 minutes, 23 seconds.

This assumes linear scaling on the Surface Laptop 4 with AMD Ryzen 5 and four CPU threads. It excludes model loading, index construction, and other processing.

Implementation considerations

- New products: Embed their search text before adding them to the searchable vector index.
- Existing products: Re-embed when text used by the encoder changes. Under your recipe, that means changes to `search_text`; changes to other fields alone do not require new vectors.
- Model changes: Re-embed the entire catalog and encode queries with the new model and its required preprocessing. Product and query vectors must use the same model version.
- Before launch: Run a fixed relevance benchmark against the new index and compare nDCG@10 and Exact@1 with the current index.
- After launch: Monitor search latency, errors, index freshness, click-through rate, add-to-cart rate, conversion rate, query reformulations, and relevance on periodically judged queries.

No embeddings were run, no test data was accessed, and no files were changed.

100 | Turn 27 | USER PROMPT

2026-10-24T01:40:25.824Z
push a PDF of this session log, with prompts, summaries, ect to this github repo.
https://github.com/u6062992/AI-Disclosures